```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os

project_path = "/content/drive/MyDrive/SensorMind"

os.makedirs(project_path, exist_ok=True)
os.makedirs(project_path + "/data", exist_ok=True)
os.makedirs(project_path + "/models", exist_ok=True)
os.makedirs(project_path + "/notebooks", exist_ok=True)
os.makedirs(project_path + "/outputs", exist_ok=True)

print("Project folders created")
```

```
Project folders created
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class AttentionBlock(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.attn = nn.Linear(hidden_size, 1)

    def forward(self, x):
        scores = self.attn(x)
        weights = F.softmax(scores, dim=1)
        context = torch.sum(weights * x, dim=1)
        return context, weights


class AttentionLSTM(nn.Module):
    def __init__(self, input_size=18, hidden_size=64, num_layers=2):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=0.2
        )

        self.attention = AttentionBlock(hidden_size)

        self.rul_head = nn.Sequential(
            nn.Linear(hidden_size, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 1)
        )

        self.stage_head = nn.Sequential(
            nn.Linear(hidden_size, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 4)   # old checkpoint = 4 classes
        )

    def forward(self, x):
```

```
        lstm_out, _ = self.lstm(x)

        context, attn_weights = self.attention(lstm_out)

        rul_out = self.rul_head(context)
        stage_out = self.stage_head(context)

        return rul_out, stage_out, attn_weights
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = AttentionLSTM()

model.load_state_dict(
    torch.load("best_attention_lstm_v2.pth", map_location=device)
)

model.eval()

print("Model loaded successfully")
```

```
Model loaded successfully
```

```
import pandas as pd
import numpy as np

df = pd.read_csv("train_engineered.csv")
```

```
feature_cols = [
    'cycle',
    's4',
    's6',
    's7',
    's9',
    's11',
    's12',
    's14',
    's15',
    's17',
    's11_roll5',
    's12_roll5',
    's6_roll5',
    's4_roll5',
    's7_roll5',
    's11_lag1',
    's12_lag1',
    's6_lag1'
]
```

```
print(len(feature_cols))
```

```
18
```

```
sequence_length = 30

X_seq = []

for unit in df['unit_id'].unique():

    unit_data = df[df['unit_id'] == unit]

    values = unit_data[feature_cols].values

    for i in range(len(values) - sequence_length):
        X_seq.append(values[i:i+sequence_length])

X_seq = np.array(X_seq)

print("X_seq shape:", X_seq.shape)
```

```
X_seq shape: (17631, 30, 18)
```

```
sample_idx = 10
```

```
sample = X_seq[sample_idx]
```

```
sample = torch.tensor(
    sample,
    dtype=torch.float32
).unsqueeze(0)

print(sample.shape)
```

```
torch.Size([1, 30, 18])
```

```
with torch.no_grad():
    rul_pred, stage_pred, attn_weights = model(sample)
```

```
predicted_rul = rul_pred.item()

predicted_stage = torch.argmax(stage_pred, dim=1).item()

attention = attn_weights.squeeze().numpy()

print("Predicted RUL:", predicted_rul)
print("Predicted Stage:", predicted_stage)
```

```
Predicted RUL: 0.45368027687072754
Predicted Stage: 1
```

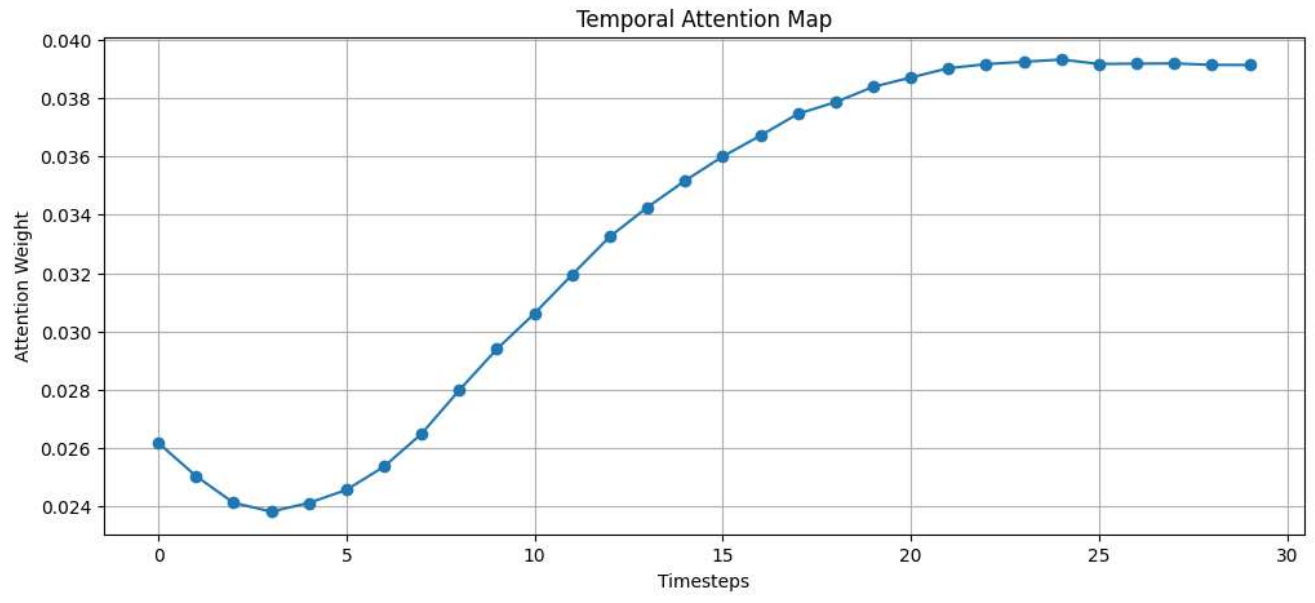
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

plt.plot(
    range(len(attention)),
    attention,
    marker='o'
)

plt.xlabel("Timesteps")
plt.ylabel("Attention Weight")
plt.title("Temporal Attention Map")

plt.grid(True)
plt.show()
```




```
top_timesteps = np.argsort(attention)[::-1][:5]

print("Critical timesteps:", top_timesteps)
```

```
Critical timesteps: [24 23 27 26 25]
```